

An Internship Report On
SECURE LOGIN SYSTEM USING CIA + AAA MODEL

Submitted in partial fulfilment of the
Requirements for the award of the Degree of
BACHELOR OF COMPUTER APPLICATIONS(BCA)

By

N.Poojitha(234122p)

Under the esteemed guidance of
SK.Johnbee M.Sc, MTech(CSE)
Assistant Professor, Department of Computer Science



DEPARTMENT OF COMPUTER SCIENCE
**PARVATHANENI BRAHMAYYA SIDDHARTHA COLLEGE OF ARTS &
SCIENCE VIJAYAWADA – 10**

An Autonomous college under the jurisdiction of Krishna University, Machilipatnam
(A college with potential for excellence, Re-accredited at the level “A+” by NAAC,
Bangalore)

2023 – 2026

**PARVATHANENI BRAHMAYYA SIDDHARTHA COLLEGE OF ARTS AND SCIENCE
VIJAYAWADA-10**

An Autonomous college under the jurisdiction of Krishna University, Machilipatnam.

(A college with potential for excellence; Re-accredited at the level 'A+' by NAAC,
Bangalore)

DEPARTMENT OF COMPUTER SCIENCE



CERTIFICATE

This is to certify that the project entitled “**SECURE LOGIN SYSTEM USING CIA +AAA MODEL**”, is a bonafide work of **N.Poojitha (234122P)** submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF SCIENCE(BCA)** from P.B. Siddhartha College of Arts and Science, Vijayawada.

SK. Johnbee M.Sc., M.Tech (CSE)

Project Guide

K. SUDHIR M.Sc (IT), M.Tech(CSE)

Head of the Department

Department of Computer Science

Internal Examiner

Date:

External Examiner

Date:



nasscom



CERTIFICATE

of COMPLETION

This is to certify that NAGULAKONDA POOJITHA
has successfully completed an internship in the
CYBER SECURITY at SandSpace Technologies Pvt Ltd
from 01/12/2025 to 07/03/2026

We acknowledge the valuable contribution made during the
internship and extend our best wishes for future achievements.



V. G. K.
HR MANAGER

ABSTRACT

I'm Nagulakonda Poojitha, a final year **BCA student from P.B. Siddhartha College of Arts & Science**. This report presents my academic project titled **"Secure Login System using CIA and AAA Security Models."** The project is developed as part of my BCA curriculum to understand and implement important cybersecurity principles in modern web applications.

The system is developed using **HTML for the frontend interface, PHP for backend processing, and MySQL for database management**.

The project applies the **CIA security model**, which focuses on maintaining **Confidentiality, Integrity, and Availability** of user data. Confidentiality is achieved through techniques such as **password hashing**, integrity is maintained through **proper input validation**, and availability is ensured by maintaining a **stable and reliable server configuration**.

In addition, the system also implements the **AAA security model**, which includes **Authentication, Authorization, and Accounting**. Authentication verifies the identity of users during login, authorization controls access permissions based on user roles, and accounting records login activities and system usage for monitoring purposes.

Through this project, I have applied theoretical cybersecurity concepts to a practical web application environment. It helps prevent common security issues such as **unauthorized access and improper data handling**. By integrating both **CIA and AAA security principles**, the project demonstrates a structured and effective approach to building a **secure and reliable login system for web applications**.

ACKNOWLEDGEMENT

With great pleasure, I take this opportunity to express my heartfelt gratitude to all the people who helped and supported me in completing this project successfully.

First of all, I would like to express my sincere gratitude to the Principal,

Dr. M. Ramesh, for giving me the opportunity and permission to carry out this project.

I express my deep gratitude and regards to **Mr. K. Sudhir**, M.S.c, M.Tech(cse), Head of the Department of Computer Science, for his encouragement and support.

I would also like to express my sincere thanks to my project guide, **SK.Johnbee**, M.S.c, M.Tech(cse), Assistant Professor, Department of Computer Science, for her valuable guidance, continuous support, and encouragement throughout the completion of this project.

I am also thankful to all the teaching and non-teaching staff of the Department of Computer Science for their support and cooperation during the course of this project.

N.Poojitha(234122p)

TABLE OF CONTENTS

S. No	Chapters	Page No
1.	Introduction	1-2
	1.1 Overview of Secure Login System	
	1.2 Importance of Cybersecurity	
	1.3 Objectives of Project	
2.	Existing System and Limitations	3
	2.1 Limitations	
3.	Proposed System	4-5
	3.1 Advantages of Proposed System	
	3.2 Features of Secure Login System	
4.	Security Models Used	6-7
	4.1 CIA Triad (Confidentiality, Integrity, Availability)	
	4.2 AAA, Model	
5.	Methodology	8-29
	5.1 Introduction	
	5.2 System Development Approach	
	5.3 Steps in Secure Login Implementation	
	5.4 Security Implementation Process	
	5.5 Summary	
6.	Hardware and Software Requirements	30
	6.1 Hardware Requirements	
	6.2 Software Requirements	
7.	Technologies Used	31
8.	System Requirement Specification	32
	8.1 Functional Requirements	
	8.2 Non-Functional Requirements	
9.	System Design	33-36
	9.1 User Registration Design	

	9.2 Login Page Design	
	9.3 User Dashboard Design	
	9.4 Admin Dashboard Design	
	9.5 Login Activity Monitoring	
10.	UML Diagrams	37-39
	10.1 Use Case Diagram	
	10.2 Class Diagram	
	10.3 Sequence Diagram	
11.	Coding and Implementation	40-46
	11.1 User Registration	
	11.2 User Login	
	11.3 Dashboard Implementation	
	11.4 Security Features	
12.	Software Testing	47-48
	12.1 Test Cases	
13.	Output Screens	49-53
	13.1 User Registration Module	
	13.2 User Login Screen	
	13.3 User Dashboard Screen	
	13.4 Admin Dashboard Screen	
	13.5 Failed Login Attempt Screen	
	13.6 Activity Log Screen	
	13.7 Security Verification Screen	
14.	Conclusion	54
15.	Future Scope	55
16.	Bibliography / Project References (Local URLs)	56

1. INTRODUCTION

In the modern digital world, most web applications require users to authenticate themselves before accessing services or sensitive information. A login system is an essential component that verifies user identity and prevents unauthorized access to protected resources. However, many traditional login systems are vulnerable to security threats such as password attacks, unauthorized access, and data breaches.

To address these issues, this project focuses on developing a Secure Login System by applying important cybersecurity principles. The system uses the CIA Security Model and the AAA Security Framework to ensure proper protection of user data and system resources. The CIA model focuses on Confidentiality, Integrity, and Availability of data, while the AAA model ensures Authentication, Authorization, and Accounting for system users.

The application is developed using HTML for the user interface, PHP for backend processing, and MySQL for database management. The XAMPP server environment is used to host and test the system locally. By integrating these technologies with security models, the project aims to provide a reliable and secure login system.

1.1 OVERVIEW OF SECURE LOGIN SYSTEM

A secure login system is a mechanism used in web applications to verify the identity of users before granting access to the system. Users enter their credentials such as username and password, which are then validated by the server using a database.

Secure login systems are widely used in many applications such as banking systems, online learning platforms, social media websites, and business management systems. These systems must implement strong security mechanisms to protect sensitive information and prevent unauthorized access.

Modern secure login systems include features such as password hashing, session management, input validation, and role-based access control. These features help protect user credentials and maintain system security.

1.2 IMPORTANCE OF CUBERSECURITY

Cybersecurity plays an important role in protecting digital systems and user data from cyber threats. As more services move to online platforms, the risk of cyber attacks such as hacking, phishing, and data breaches has increased significantly.

Login systems are often targeted by attackers because they store sensitive user information such as usernames and passwords. If these systems are not properly secured, attackers can gain unauthorized access and misuse the data.

Implementing strong cybersecurity practices helps prevent such risks. Security models like the CIA triad and AAA framework provide structured methods for protecting information, controlling access, and monitoring user activities.

1.3 OBJECTIVES OF PROJECT

The main objective of this project is to design and develop a secure login system that follows modern cybersecurity practices.

The specific objectives include:

- To develop a web-based secure login application.
- To implement the CIA security model to protect data.
- To apply the AAA framework for authentication and access control.
- To store user credentials securely using password hashing.
- To implement role-based authorization for different users.
- To maintain activity logs for monitoring user actions

2. EXISTING SYSTEM AND LIMITATIONS

Many traditional login systems used in early web applications were developed without strong security mechanisms. These systems mainly focused on basic login functionality rather than implementing proper cybersecurity practices. As a result, they contain several weaknesses that make them vulnerable to cyber threats.

LIMITATIONS

Plain Text Password Storage

Many traditional systems store user passwords directly in the database without encryption or hashing. If the database is compromised, attackers can easily access and misuse the passwords of users.

Lack of Role-Based Authorization

Most existing systems do not implement role-based access control. This means users may get access to system features that they are not authorized to use.

No Activity Monitoring

Traditional login systems often do not maintain records of user activities such as login time, logout time, or IP address. Without these logs, it becomes difficult to track suspicious behaviour.

Vulnerability to Cyber Attacks

Due to weak security mechanisms, these systems are more vulnerable to cyber-attacks such as SQL injection, password guessing, and unauthorized access.

Weak Input Validation

Many systems do not properly validate user inputs, which may allow attackers to manipulate login fields and exploit system vulnerabilities.

3. PROPOSED SYSTEM

The proposed system is a **Secure Login System** designed using the **CIA security model** and the **AAA framework** to improve the security of user authentication. The system focuses on protecting user information, controlling access to system resources, and monitoring user activities. By applying these cybersecurity principles, the system reduces the chances of unauthorized access and protects sensitive data.

In this system, users first register by providing their credentials. The passwords entered by users are stored in the database using hashing techniques to ensure security. During the login process, the system verifies the entered credentials and allows access only if they match the stored information.

The system also includes role-based access control, which allows administrators and regular users to access different features based on their permissions. In addition, the system records login details such as login time and IP address to maintain accountability and monitor system usage.

3.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed secure login system offers several advantages over traditional login systems. These advantages improve the security, reliability, and efficiency of the system.

Enhanced Data Security

The system uses password hashing techniques to protect user credentials. Instead of storing passwords in plain text, they are converted into encrypted hash values, which prevents attackers from easily retrieving the original passwords.

Role-Based Access Control

The system provides different levels of access for administrators and regular users. This ensures that users can only access features that are permitted for their role, reducing the risk of unauthorized access.

Improved Monitoring and Accountability

The system maintains records of user activities such as login time, IP address, and login attempts. This information helps administrators monitor system usage and detect suspicious behaviour.

Protection Against Common Cyber Threats

By implementing secure coding practices and security models, the system reduces vulnerabilities such as unauthorized access and password attacks.

Better System Reliability

The use of structured cybersecurity models ensures that the system operates in a stable and secure manner. It improves the overall trustworthiness of the application.

User-Friendly Interface

The system provides simple and easy-to-use login and registration pages, allowing users to interact with the system without difficulty.

3.2 FEATURES OF SECURE LOGIN SYSTEM

The Secure Login System includes several features that improve both system functionality and security.

User Registration Module

The system allows new users to register by entering their username and password. The registration process stores the user details securely in the database.

Secure Login Authentication

Users must enter valid credentials to access the system. The system verifies the entered username and password before granting access.

Password Hashing

Passwords are encrypted using hashing functions before being stored in the database. This prevents attackers from directly reading user passwords.

Role-Based Authorization

The system assigns different roles to users, allowing administrators and normal users to access different system features based on their permissions.

Activity Logging

The system records login activities such as login time, IP address, and user actions. These logs help monitor system usage and detect unusual activity.

Session Management

The system manages user sessions to maintain a secure login state and prevent unauthorized access during user interaction.

Database Security

User credentials and activity logs are stored securely in a structured database, which ensures better data management and protection.

4. SECURITY MODELS USED

To enhance the security of the login system, two important cybersecurity frameworks are used in this project: the **CIA Security Model** and the **AAA Security Model**. These models help protect sensitive user data, control access to system resources, and monitor user activities. By implementing these security models, the system ensures that user credentials remain secure and only authorized users can access the application.

The CIA model focuses on protecting data through three main principles: confidentiality, integrity, and availability. The AAA model focuses on user authentication, authorization, and accounting of activities. Together, these models create a strong security foundation for the secure login system.

4.1 CIA TRIAD (CONFIDENTIALITY, INTEGRITY, AVAILABILITY).

Confidentiality

Confidentiality ensures that sensitive information is accessible only to authorized users. In the secure login system, confidentiality is maintained by protecting user credentials using password hashing techniques. This prevents unauthorized users from viewing or accessing confidential data stored in the database.

Integrity

Integrity ensures that data remains accurate and unchanged unless modified by authorized users. In this system, data integrity is maintained by validating user inputs and controlling database operations. This helps prevent unauthorized modifications to stored information.

Availability

Availability ensures that system resources and services remain accessible to authorized users whenever required. In this project, the system is hosted on a stable server environment using Apache and MySQL through the XAMPP platform, ensuring that the login system is available for users.

4.2 AAA MODEL (AUTHENTICATION, AUTHORIZATION, ACCOUNTING)

The **AAA Model** is another important security framework used to control user access and track system activities. It consists of three main components: Authentication, Authorization, and Accounting.

Authentication

Authentication is the process of verifying the identity of a user before allowing access to the system. In the secure login system, authentication is performed by validating the username and password entered by the user.

Authorization

Authorization determines what actions a user can perform after successful authentication. The system uses role-based access control to provide different permissions for administrators and regular users.

Accounting

Accounting refers to recording and monitoring user activities within the system. The system maintains logs of login activities, including login time and IP address. These records help administrators track system usage and detect suspicious activities.

By implementing the AAA model, the system ensures that users are properly verified, given appropriate permissions, and their activities are recorded for security purposes.

5. METHODOLOGY

5.1 INTRODUCTION

This chapter describes the methodology followed for the design and development of the **Secure Login System based on the CIA security model and the AAA framework**. The purpose of this methodology is to provide a structured approach for implementing a secure authentication system that protects user information and system resources. The system is developed using web technologies such as **HTML** and **PHP**, with **MySQL** used as the

database and **XAMPP** as the server environment.

The methodology focuses on implementing important cybersecurity principles including Confidentiality, Integrity, and Availability from the CIA model, along with Authentication, Authorization, and Accounting from the AAA framework. These principles help ensure secure user access, protection of sensitive data, and proper monitoring of system activities.

The development process involves several stages such as system setup, database design and creation, implementation of login and registration modules, integration of security mechanisms, and system testing. By following this structured methodology, the system aims to provide a reliable, secure, and efficient login environment for both administrators and users.

5.2 SYSTEM DEVELOPMENT APPROACH

5.2.1 System Setup

The development environment was prepared by installing XAMPP, which provides Apache server and MySQL database support.

The Apache and MySQL services were started from the XAMPP control panel to enable local server execution.

➤ Open XAMPP Control Panel Click:

- Start Apache
- Start MySQL

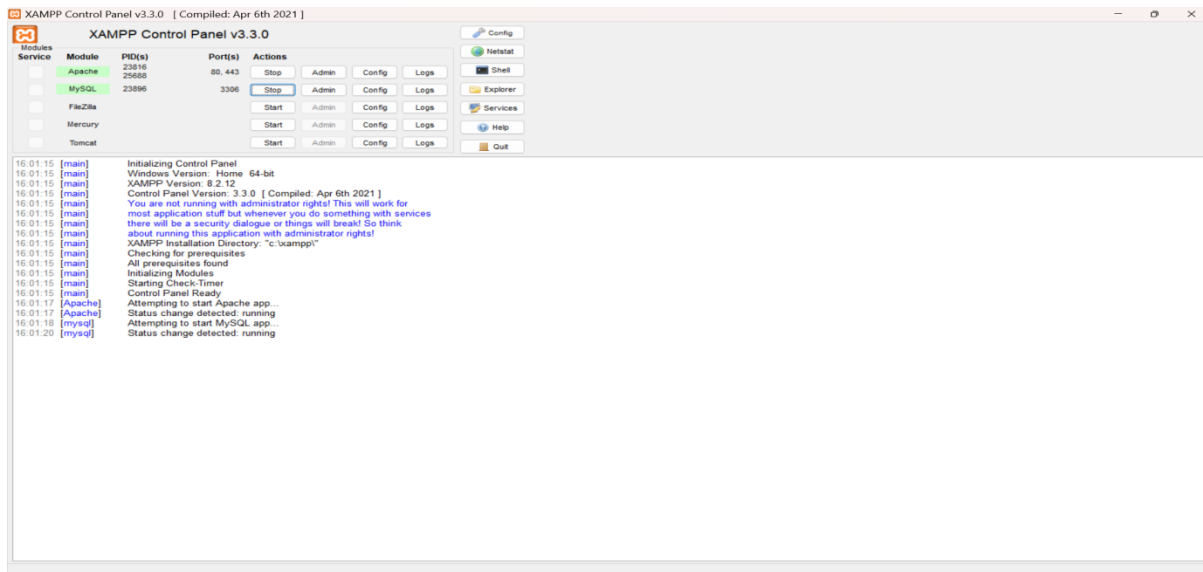


Figure 5.1: XAMPP Control Panel Showing Apache and MySQL Running

5.2.2 Project Structure

A project folder named **secure_login** was created inside the *htdocs* directory of XAMPP. The following files were created for system implementation:

- config.php
- register.php
- login.php
- dashboard.php
- admin_dashboard.php
- logout.php

These files collectively manage database connection, authentication, authorization, and logging mechanisms.

➤ Create Project Folder

- 1. Open this location in your computer: C:\xampp\htdocs\
- 2. Create a folder: folder name: secure_login

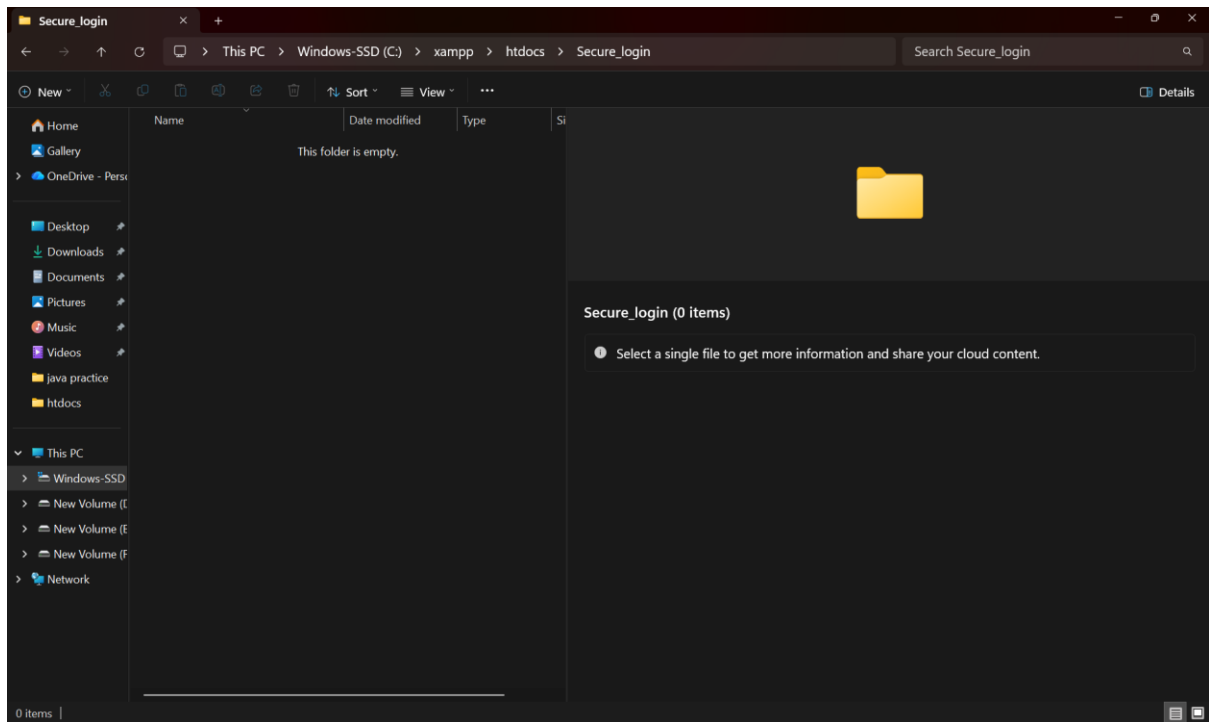


Figure 5.2: Project Folder Structure in htdocs Directory

5.2.3 Database Creation

The database was created using phpMyAdmin interface.

➤ Creation of Database

A database named **secure_login_db** was created to store user credentials and activity logs.

- Open browser: <http://localhost/phpmyadmin>
- Database name: secure_login_db

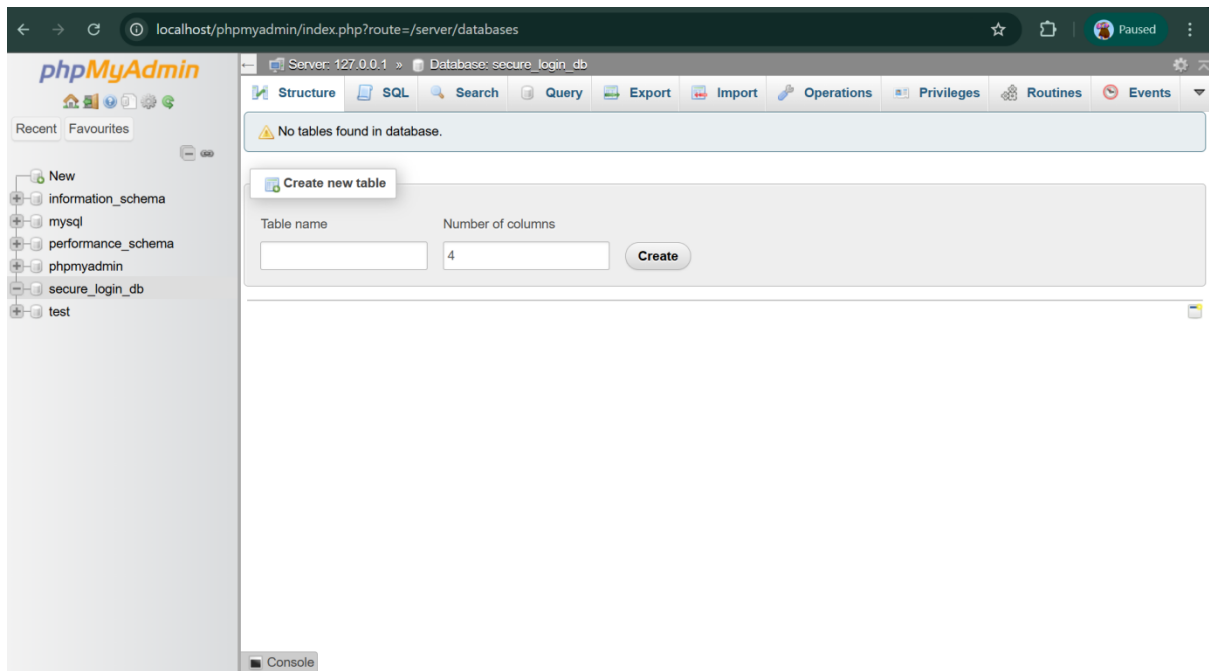


Figure 5.3: Database Creation in phpMyAdmin

5.2.4 Users Table Creation

A table named **users** was created to store user account details. The structure includes:

- id (Primary Key)
- username (Unique)
- password (Hashed)
- role (admin/user)
- created_at (Timestamp)

The role field is used for implementing authorization.

➤ **Create Users Table:**

- Click **secure_login_db**
- **Go to the SQL tab**
- Paste this code:

```
CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY,username VARCHAR(50) UNIQUE,password VARCHAR(255),role VARCHAR(10),created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

- Click go

The screenshot shows the phpMyAdmin interface for the 'secure_login_db' database, specifically the 'users' table structure. The table has the following columns:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	username	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	password	varchar(255)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	role	enum('admin', 'user')	utf8mb4_general_ci		Yes	NULL			Change Drop More
5	created_at	timestamp			No	current_timestamp()			Change Drop More

Below the table structure, there is an 'Indexes' section showing the primary key on the 'id' column.

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	
Edit Rename Drop	username	BTREE	Yes	No	username	0	A	Yes	

Figure 5.4: Users Table Structure

5.2.5 Logs Table Creation (Accounting Mechanism)

A table named **logs** was created to record user activities. The table includes:

- log_id (Primary Key)
- username
- action
- login time
- Ip address

This table implements the accounting component of the AAA model.

➤ **Create Logs Table:**

- Click **SQL** again and paste:

```
CREATE TABLE logs (log_id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR (50), action VARCHAR (50), login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP, ip_address VARCHAR (50));
```
- Click **Go**

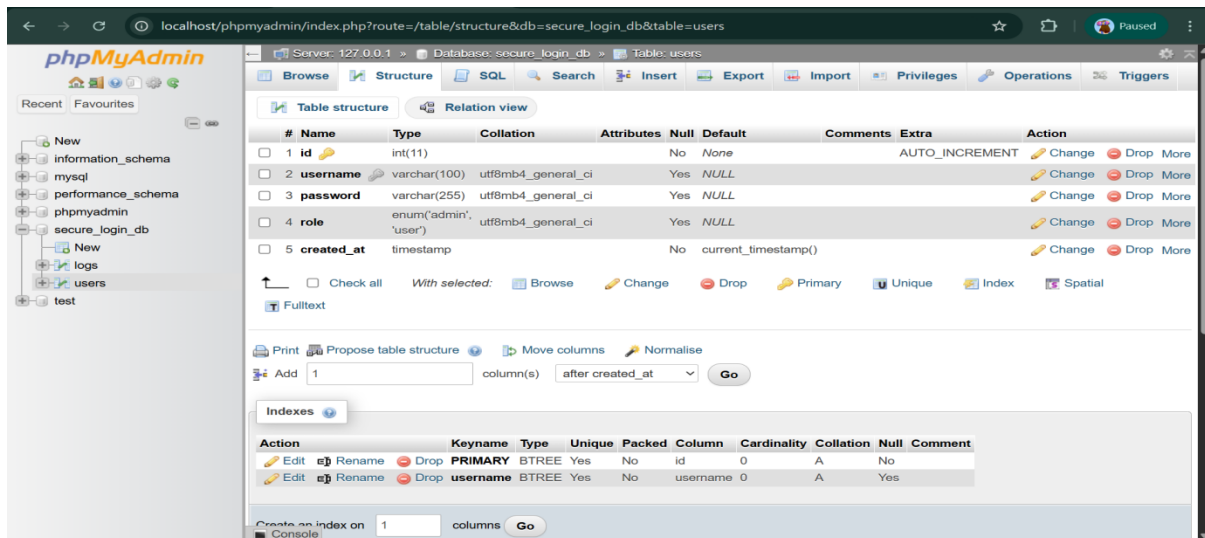


Figure 5.5: Logs Table Structure

5.2.6 Database Connectivity

The connection between the application and database was established using the config.php file with MySQLi connection method. This ensures all application pages can securely interact with the database.

➤ **Database Connection:**

- Create **config.php** in a secure_login folder
- Paste this code:

```
<?php

$servername = "localhost";

$username = "root";

$password = "";

$dbname = "secure_login_db";

$conn = new mysqli($servername,$username,$password,$dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

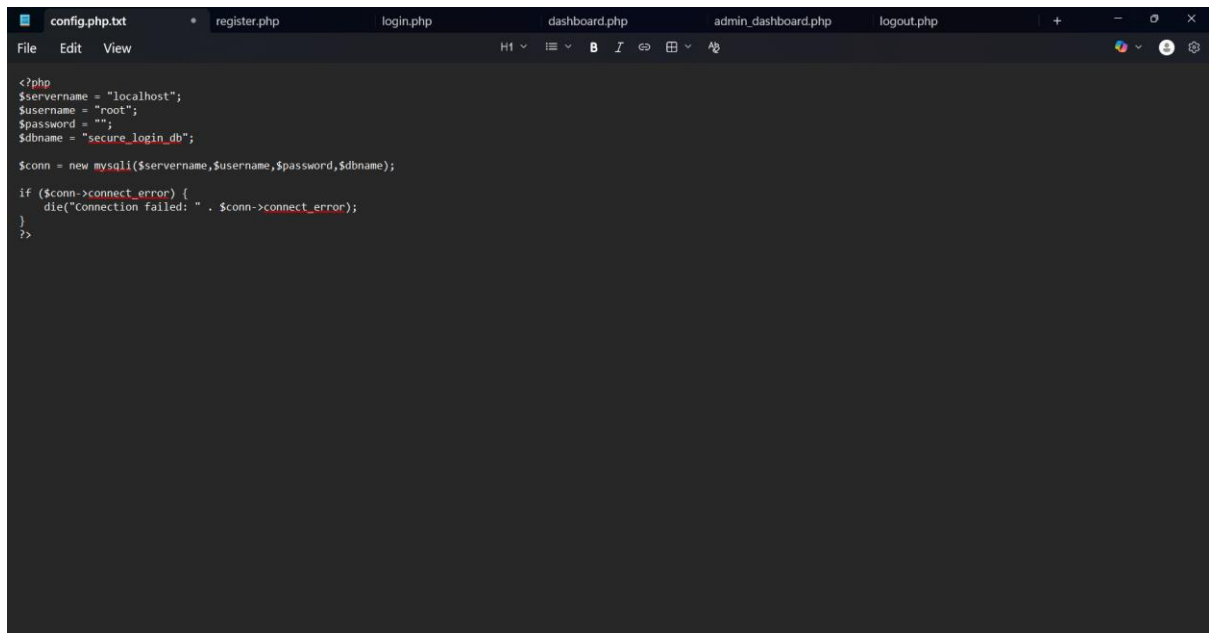


Figure 5.6: Database Connection Code (config.php)

➤ **Register Page:**

- Create **register.php** in secure_login folder
- Paste this code:

```
<?php
include("config.php");

if(isset($_POST['register'])){

$username = $_POST['username'];
$password = password_hash($_POST['password'], PASSWORD_DEFAULT);
$role = $_POST['role'];

$sql = "INSERT INTO users(username,password,role)
VALUES('$username','$password','$role')";

if(mysqli_query($conn,$sql)){
    echo "Registration Successful";
}else{
    echo "Error: " . mysqli_error($conn);
}

}

?>

<!DOCTYPE html>
<html>
<head>
```

```

<title>Register</title>
</head>
<body>

<h2>Registration Page</h2>

<form method="POST">
Username: <input type="text" name="username" required><br><br>
Password: <input type="password" name="password" required><br><br>

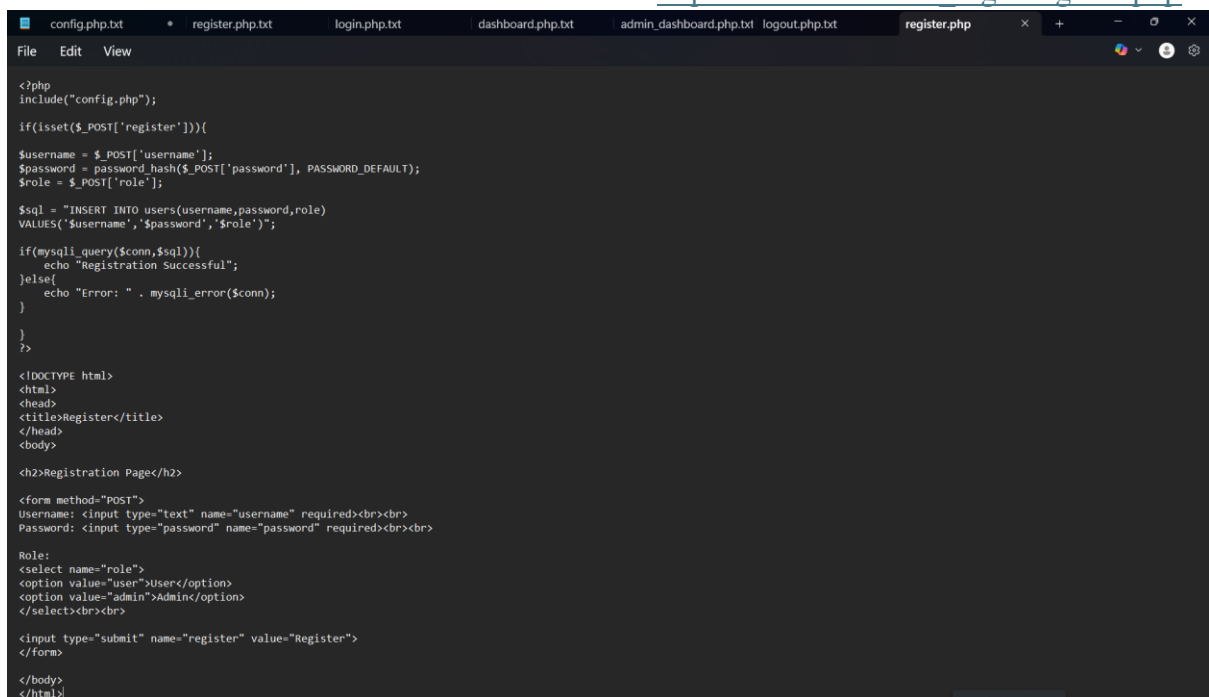
Role:
<select name="role">
<option value="user">User</option>
<option value="admin">Admin</option>
</select><br><br>

<input type="submit" name="register" value="Register">
</form>

</body>
</html>

```

- To Run this code: Click browser: http://localhost/secure_login/register.php



```

<?php
include("config.php");

if(isset($_POST['register'])){
    $username = $_POST['username'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
    $role = $_POST['role'];

    $sql = "INSERT INTO users(username,password,role)
    VALUES('$username','$password','$role')";

    if(mysqli_query($conn,$sql)){
        echo "Registration Successful";
    }else{
        echo "Error: " . mysqli_error($conn);
    }
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Register</title>
</head>
<body>

<h2>Registration Page</h2>

<form method="POST">
Username: <input type="text" name="username" required><br><br>
Password: <input type="password" name="password" required><br><br>

Role:
<select name="role">
<option value="user">User</option>
<option value="admin">Admin</option>
</select><br><br>

<input type="submit" name="register" value="Register">
</form>

</body>
</html>

```

Figure 5.7: Registration Page (register.php)

➤ Login Page:

- Create login.php

- Paste this code

```
<?php
session_start();
include "config.php";

if(isset($_POST['login']))
{
$username=$_POST['username'];
$password=$_POST['password'];

$sql="SELECT * FROM users WHERE username='$username'";
$result=$conn->query($sql);

if($result->num_rows>0)
{
$row=$result->fetch_assoc();

if(password_verify($password,$row['password']))
{

$_SESSION['username']=$username;
$_SESSION['role']=$row['role'];

$ip=$_SERVER['REMOTE_ADDR'];

$conn->query("INSERT INTO logs(username,action,ip_address)
VALUES('$username','login','$ip')");

if($row['role']=="admin")
{
header("Location: admin_dashboard.php");
```

```
}  
else  
{  
header("Location: dashboard.php");  
}
```

```
}  
else  
{  
echo "Invalid Password";  
}
```

```
}  
else  
{  
echo "User not found";  
}
```

```
}  
?>
```

```
<form method="POST">
```

```
<h2>Login</h2>
```

Username:

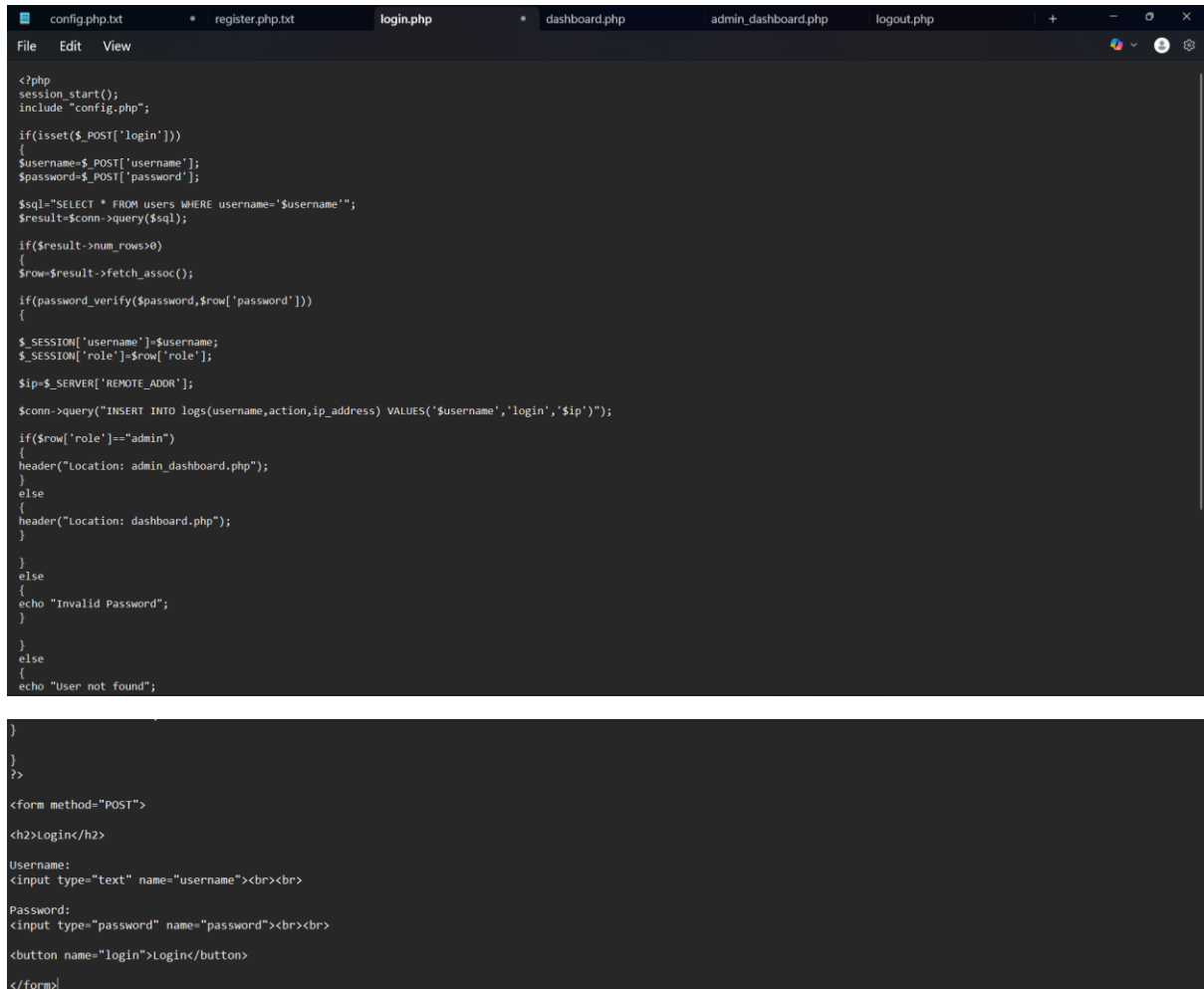
```
<input type="text" name="username"><br><br>
```

Password:

```
<input type="password" name="password"><br><br>
```

```
<button name="login">Login</button>
```

```
</form>
```



```
<?php
session_start();
include "config.php";

if(isset($_POST['login']))
{
    $username=$_POST['username'];
    $password=$_POST['password'];

    $sql="SELECT * FROM users WHERE username='$username'";
    $result=$conn->query($sql);

    if($result->num_rows>0)
    {
        $row=$result->fetch_assoc();

        if(password_verify($password,$row['password']))
        {
            $_SESSION['username']=$username;
            $_SESSION['role']=$row['role'];

            $ip=$_SERVER['REMOTE_ADDR'];

            $conn->query("INSERT INTO logs(username,action,ip_address) VALUES('$username','login','$ip')");

            if($row['role']=="admin")
            {
                header("Location: admin_dashboard.php");
            }
            else
            {
                header("Location: dashboard.php");
            }
        }
        else
        {
            echo "Invalid Password";
        }
    }
    else
    {
        echo "User not found";
    }
}

?>

<form method="POST">

<h2>Login</h2>

Username:
<input type="text" name="username"><br><br>

Password:
<input type="password" name="password"><br><br>

<button name="login">Login</button>

</form>
```

Figure 5.8: Login Page (login.php)

➤ **User Dashboard:**

- Create dashboard.php
- Paste this code

```
<?php
```

```
session_start();
```

```
if(!isset($_SESSION['username']))
```

```
{
```

```
header("Location: login.php");  
}  
?>
```

```
<h2>User Dashboard</h2>
```

```
Welcome <?php echo $_SESSION['username']; ?>
```

```
<br><br>
```

```
<a href="logout.php">Logout</a>
```

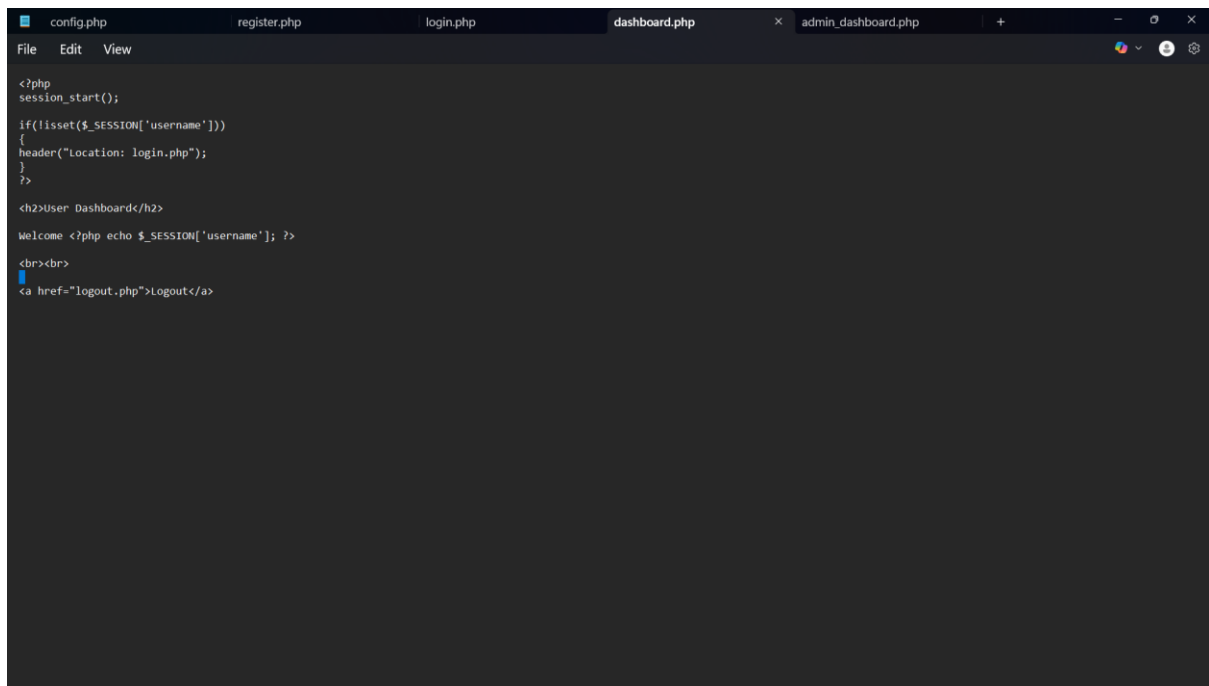


Figure 5.9: User Dashboard (dashboard.php)

➤ **Admin Dashboard:**

- Create admin_dashboard.php
- Paste this code

```
<?php  
session_start();
```

```
if($_SESSION['role']!="admin")
{
header("Location: login.php");
}
?>
```

<h2>Admin Dashboard</h2>

Welcome Admin <?php echo \$_SESSION['username']; ?>

Logout

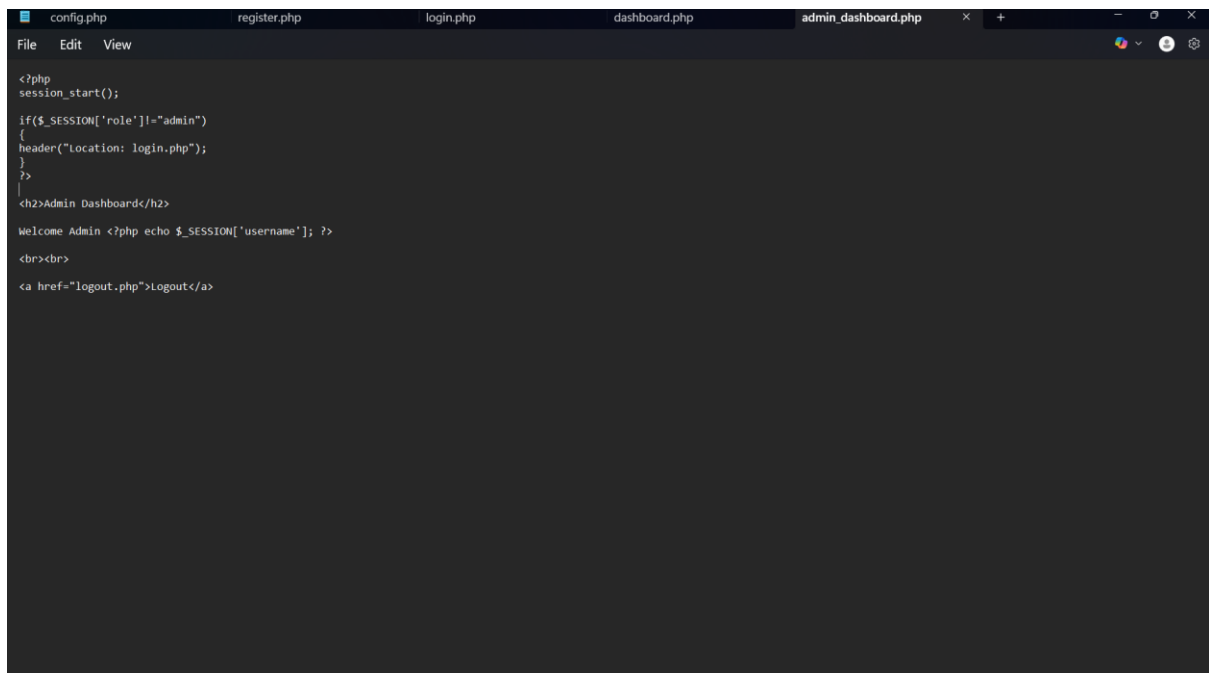


Figure 5.10: Admin Dashboard (admin_dashboard.php)

➤ **Logout File:**

- Create logout.php
- Paste this code

```
<?php
session_start();
```

```
session_destroy();  
header("Location: login.php");  
?>
```

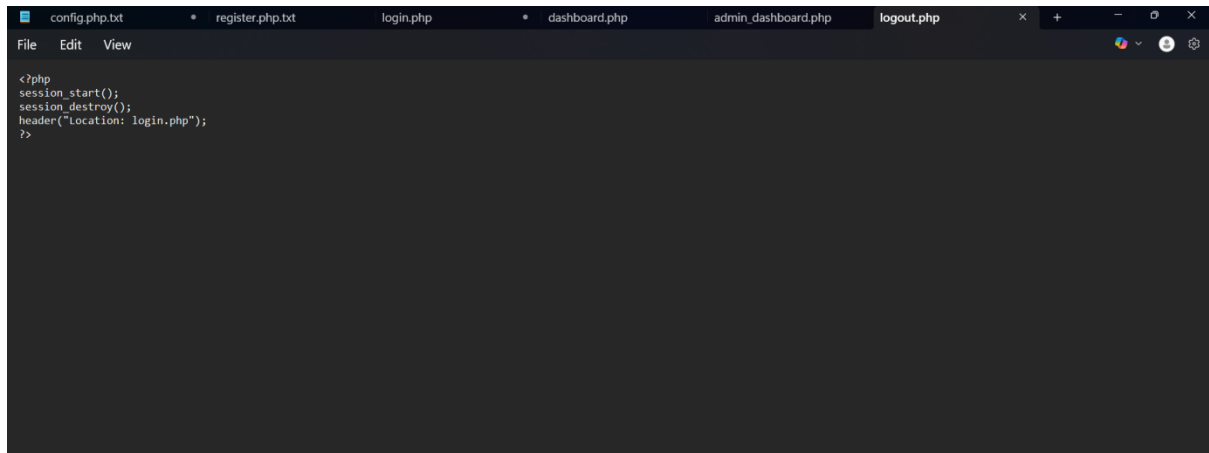


Figure 5.11:logout file (logout.php)

5.3 STEPS IN SECURE LOGIN IMPLEMENTATION

Authentication Implementation

Authentication verifies the identity of users using username and password.

5.3.1 User Registration

During registration, the password entered by the user is encrypted using PHP's password_hash() function before storing in the database. This ensures confidentiality of user credentials.

- To Run this code: Click browser: http://localhost/secure_login/register.php
- Test Registration
- Register 2 users.

Example:

1. Username: Poojitha

Password: 22341

Role: admin

2. Username: Pandu

Password: 14322

Role: user

Registration Page

Username:

Password:

Role: User

Figure 5.12: User Registration Page

Showing rows 0 - 1 (2 total, Query took 0.0003 seconds.)

```
SELECT * FROM `users`
```

Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	id	username	password	role	created_at
☐	1	Poojitha	\$2y\$10\$XOBeeZ3J7TPn9XIORAut.EM1EFqoXnksNUqo.qP105...	admin	2026-03-06 20:38:30
☐	2	Pandu	\$2y\$10\$fAeICf3j0KErE0j55FLudeZbsWGEpu1ytlzSYRekH4...	user	2026-03-06 20:38:45

Query results operations: [Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

Figure 5.13: Hashed Password Stored in Database

5.3.2 Login Verification

During login, the system validates the entered password using `password_verify()` function. If the credentials are correct, the user is redirected to the appropriate dashboard.

- To Run this code: Click browser: http://localhost/secure_login/login.php
- **Test Login**
- **Correct Login**
Login with: Poojitha(username)
22341(password)



Figure 5.14: Login Page Interface

- Successful Login Redirect to Dashboard

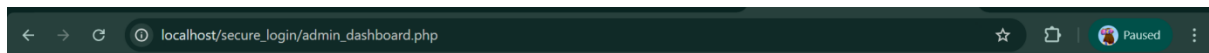


Figure 5.15: Successful Login Redirect to Dashboard

➤ **Wrong Login:**

- Enter wrong password.

Example:

Username: Poojitha

Password: 11111

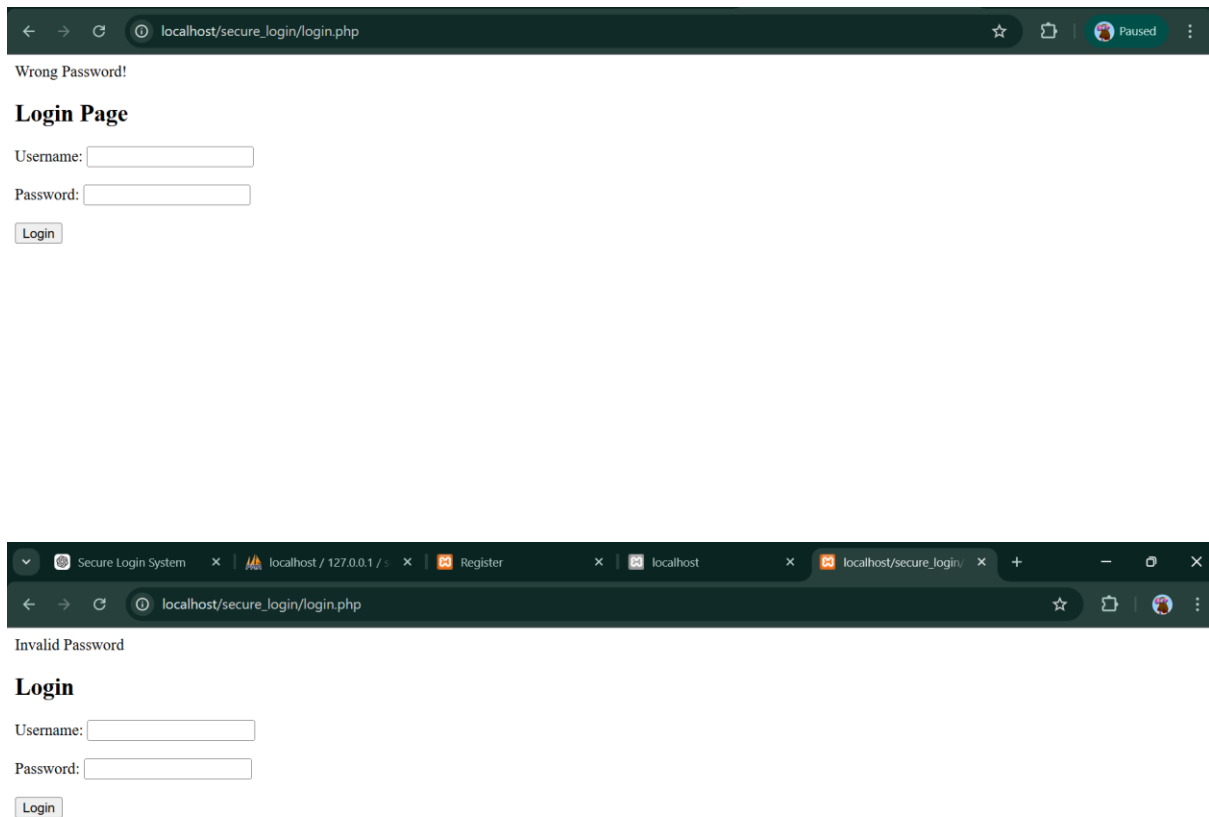


Figure 5.16: Failed Login Attempt

5.3.3 Authorization Implementation

Authorization is implemented using role-based access control. After successful login:

- Users with role “admin” are redirected to admin_dashboard.php
- Users with role “user” are redirected to dashboard.php

Unauthorized access attempts are blocked using session validation.

➤ Test Authorization:

1.Admin Login

Login as: Poojitha

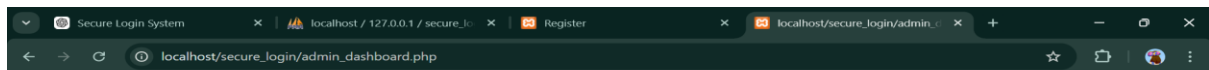
You will see: admin_Dashboard.php



Login Page

Username:

Password:

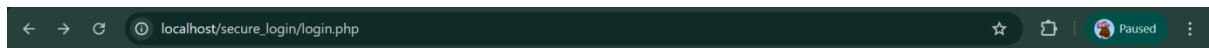


Admin Dashboard

Welcome Admin Poojitha

[Logout](#)

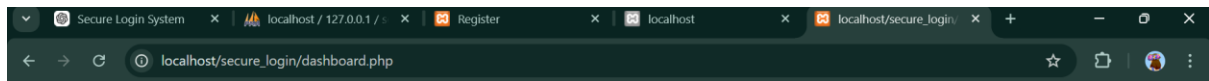
Figure 5.17: Admin Dashboard Interface



Login Page

Username:

Password:



User Dashboard

Welcome Pandu

[Logout](#)

Figure 5.18: User Dashboard Interface



Figure 5.19: Unauthorized Access Redirection

5.4 SECURITY IMPLEMENTATION PROCESS

5.4.1 Accounting Implementation

The Accounting component records user activities such as login time and IP address. After successful authentication, user activity is stored in the logs table.

This mechanism helps monitor system usage and enhances security auditing.

➤ **Test Accounting (Logs):**

- Login multiple times.
- Then open: phpMyAdmin → logs → Browse
- You will see:

username
actions
login_time
ip_address

Showing rows 0 - 5 (6 total, Query took 0.0006 seconds.)

SELECT * FROM `logs`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	log_id	username	action	login_time	ip_address
☐ Edit ☐ Copy ☐ Delete	1	Poojitha	login	2026-03-06 20:48:34	::1
☐ Edit ☐ Copy ☐ Delete	2	Pandu	login	2026-03-06 20:49:36	::1
☐ Edit ☐ Copy ☐ Delete	3	Poojitha	login	2026-03-06 20:56:09	::1
☐ Edit ☐ Copy ☐ Delete	4	Pandu	login	2026-03-06 20:56:25	::1
☐ Edit ☐ Copy ☐ Delete	5	Poojitha	login	2026-03-06 20:56:36	::1
☐ Edit ☐ Copy ☐ Delete	6	Pandu	login	2026-03-06 20:56:48	::1

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

Figure 5.20: Logs Table Showing User Login Records

5.4.2 Implementation of CIA Security Model

1. Confidentiality

Confidentiality is maintained through:

- Password hashing using password_hash()
- Secure session management
- Role-based access control

2. Integrity

Integrity is ensured by:

- Unique username constraint
- Proper password verification
- Controlled database operations

3. Availability

Availability is ensured by:

- Running Apache and MySQL services
- Stable local server configuration

- Proper error handling

5.4.3 Mapping of CIA and AAA Implementation

Security Feature	CIA Principle	AAA Principle
Password Hashing	Confidentiality	Authentication
Role-Based Access	Confidentiality	Authorization
Login Logs	Integrity	Accounting
SQL Injection Prevention	Integrity	-
Session Management	Availability + Integrity	Authorization

The implementation aligns with security principles as follows:

5.5 SUMMARY

This chapter explained the methodology used for developing the Secure Login System based on the CIA security model and the AAA framework. The system was developed using web technologies such as HTML and PHP, with MySQL used as the database and XAMPP as the server environment.

The methodology included important stages such as system setup, project structure organization, database creation, and implementation of user authentication and authorization mechanisms. Modules such as user registration, login verification, user dashboard, admin dashboard, and logout functionality were implemented to ensure proper system operation.

Security mechanisms were integrated throughout the system to protect user data and maintain system reliability. The principles of Confidentiality, Integrity, and Availability ensured data protection, while Authentication, Authorization, and Accounting controlled user access and recorded login activities.

Overall, the methodology provided a structured approach for developing a secure and reliable login system that supports controlled access and effective monitoring of user activities.

6. HARDWARE AND SOFTWARE REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

Hardware requirements refer to the physical components needed to develop and run the Secure Login System. A computer system with basic specifications is sufficient for the development and execution of this project.

Minimum Hardware Requirements:

- Processor: Intel Core i3 or higher
- RAM: 4 GB or above
- Hard Disk: 500 GB or higher
- Monitor: Standard display monitor
- Keyboard and Mouse: Required for user interaction

These hardware components provide the necessary support for developing, running, and testing the secure login system.

6.2 SOFTWARE REQUIREMENTS

Software requirements include the programs and tools needed to develop and operate the Secure Login System.

Required Software:

- Operating System: Windows 10 or above
- Web Server: XAMPP Server
- Programming Languages: HTML and PHP
- Database: MySQL
- Web Browser: Google Chrome or Mozilla Firefox
- Code Editor: Visual Studio Code or Notepad++

These software tools help in designing the web interface, implementing server-side logic, managing the database, and testing the system.

7. TECHNOLOGIES USED

The Secure Login System is developed using basic web technologies that support user authentication and database management.

HTML:

HTML is used to design the structure of the web pages such as the registration form, login page, and dashboard pages. It helps in creating the user interface elements that allow users to interact with the system.

PHP:

PHP is used as the server-side programming language to handle the system logic. It processes user input, verifies login credentials, communicates with the database, and controls access to different pages based on user roles.

MySQL:

MySQL is used as the database management system to store user information and login activity logs. It helps in securely managing data and retrieving information whenever required.

XAMPP:

XAMPP provides a local server environment that includes Apache and MySQL services. It allows the system to run and be tested on a local machine during the development process.

8. SYSTEM REQUIREMENTS SPECIFICATION

The System Requirement Specification (SRS) defines the requirements needed for the proper functioning of the Secure Login System. It describes both the functional and non-functional aspects of the system.

8.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the main operations performed by the system.

- The system must allow users to register by entering their details.
- The system must allow registered users to log in using valid credentials.
- The system must verify the user's identity before granting access.
- The system must provide separate dashboards for users and administrators.
- The system must record login activities in the database.
- The system must allow users to log out securely.

8.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality and performance characteristics of the system.

- **Security:** The system must protect user information through secure authentication.
- **Performance:** The system should respond quickly during login and page access.
- **Reliability:** The system should function properly without frequent errors.
- **Usability:** The interface should be simple and easy for users to understand.

9. SYSTEM DESIGN

This chapter presents the overall design of the **Secure Login System**, which has been developed using **HTML**, **PHP**, and **MySQL**. System design is a critical phase in software development as it defines the organization and interaction of the system's components. A well-structured design ensures that the application functions efficiently, is easy to maintain, and meets the required security and usability objectives.

The Secure Login System is specifically designed to provide **secure user authentication**, **controlled access** to system resources, and **tracking of login activities**. The system is modular, consisting of several key components: **user registration**, **login verification**, **user dashboard**, **admin dashboard**, and **logout functionality**. These modules work in coordination to ensure that only authorized users can access specific system features, thereby maintaining security and integrity.

The system employs a **MySQL database** to securely store user credentials and login activity logs. **PHP** is used to establish the connection between the web interface and the database, performing essential operations like authentication, authorization, and data validation. Additionally, the design incorporates crucial security principles based on the **CIA (Confidentiality, Integrity, Availability) model** and the **AAA (Authentication, Authorization, Accounting) framework** to safeguard user access and monitor system activity.

9.1 USER REGISTRATION DESIGN

The **user registration module** enables new users to create an account within the system. During registration, users are required to provide details such as a **username** and **password**. Once the registration form is submitted, the system validates the input and securely stores the user information in the database. Proper input validation and encryption ensure that sensitive data remains protected.

A screenshot of a web browser displaying a registration page. The browser's address bar shows 'localhost/secure_login/register.php'. The page title is 'Registration Page'. It contains three input fields: 'Username:', 'Password:', and 'Role:'. The 'Role:' field is a dropdown menu currently showing 'User'. Below these fields is a 'Register' button.

← → ↻ localhost/secure_login/register.php ☆ 📄 ⌵ Paused ⋮

Registration Page

Username:

Password:

Role: ▾

Figure 9.1: User Registration Page

9.2 LOGIN PAGE DESIGN

The **login page** allows registered users to securely access the system. Users enter their credentials, which the system verifies against the stored data in the database. Only after successful verification is the user granted access. The login mechanism also includes measures such as password hashing and error handling to prevent unauthorized access and enhance system security.



Figure 9.2: Login Page Interface

9.3 USER DASHBOARD DEIGN

Once a user successfully logs in, they are redirected to the **user dashboard**. This interface provides a personalized view, displaying user-specific information and options to interact with the system. The dashboard is designed to be intuitive and responsive, ensuring that users can easily navigate and perform allowed actions without compromising system security.

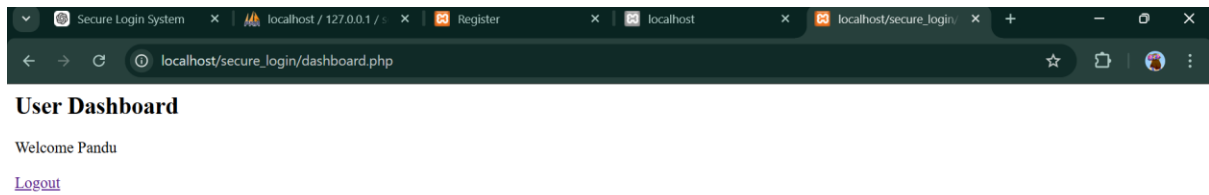


Figure 9.3: User Dashboard

9.4 ADMIN DASHBOARD DESIGN

The **admin dashboard** provides administrators with tools to manage the system and monitor user activity. Admins can view detailed information about registered users, check login history, and perform administrative tasks such as account management. This module is crucial for maintaining system oversight and ensuring that unauthorized activities are detected promptly.

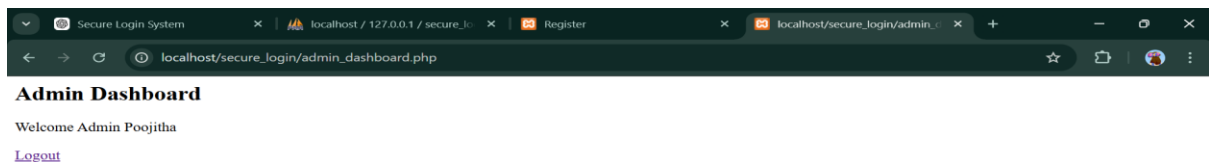


Figure 9.4: Admin Dashboard

9.5 LOGIN ACTIVITY MONITORING

To ensure accountability and maintain system integrity, the system **records all login activities** in the database. These logs capture details such as login time, user identity, and access actions. Administrators can review these records to monitor user activity, detect suspicious behavior,

and ensure that the system is being used appropriately. This logging mechanism is an essential part of the system’s security framework.

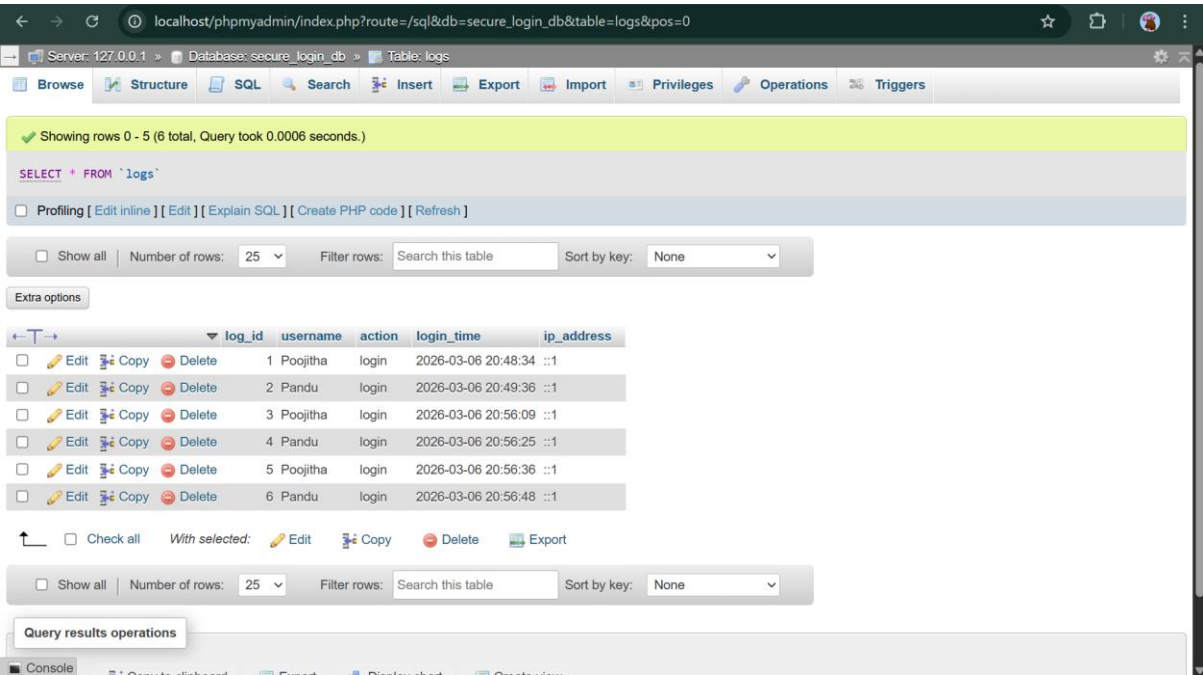


Figure 9.5: Login Activity Logs

10. UML DIAGRAMS

UML (Unified Modelling Language) diagrams provide a **visual representation of the structure, behaviour, and interactions** within the Secure Login System. These diagrams help developers, testers, and stakeholders clearly understand how the system components work together, identify potential issues early, and serve as a blueprint for system development and implementation. By using UML, the design of the system becomes more structured, maintainable, and easier to communicate.

10.1 USE CASE DIAGRAM

The **Use Case Diagram** demonstrates the interaction between the actors (users and administrators) and the system. It shows the **functional requirements** and how different types of users perform specific operations within the system.

Key operations include:

- **User:** Register an account, log in, access the user dashboard, and log out.
- **Admin:** Log in, manage users, view login activity, and audit system logs.

The diagram helps in **identifying all system functionalities from the perspective of the users**, ensuring that every required action is captured and correctly implemented.

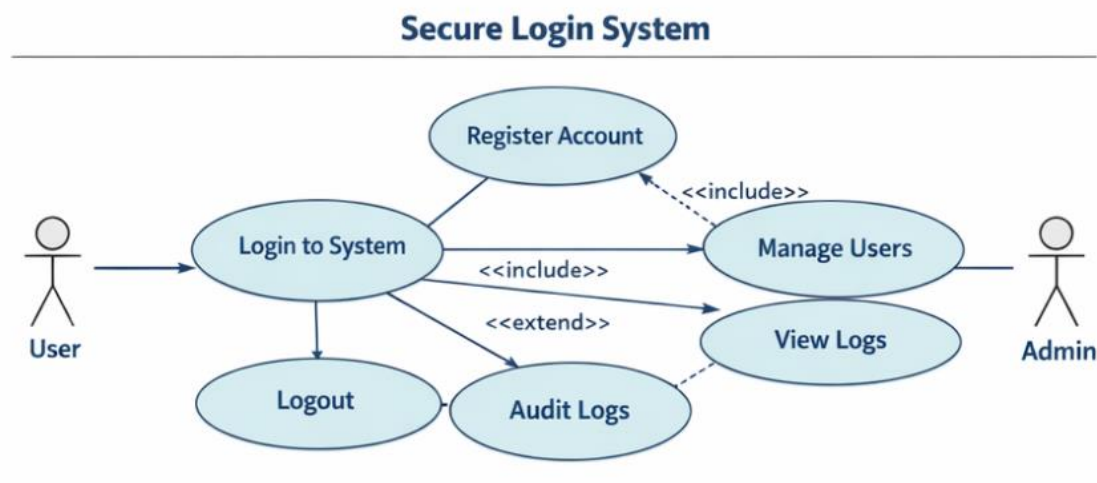


Figure 10.1: Use Case Diagram

Figure 10.1: Use Case Diagram

10.2 CLASS DIAGRAM

The **Class Diagram** represents the **static structure of the system** by showing system classes, their attributes, methods, and relationships. Key classes in the Secure Login System include:

- **User:** Stores user details and handles registration.
- **Admin:** Manages user accounts and system monitoring.
- **Login:** Handles authentication, session creation, and login validation.

- **ActivityLog:** Records user login activity and stores audit information.

The class diagram provides a **clear blueprint of system components**, showing how they interact with each other and ensuring that all entities, data, and methods are organized efficiently.

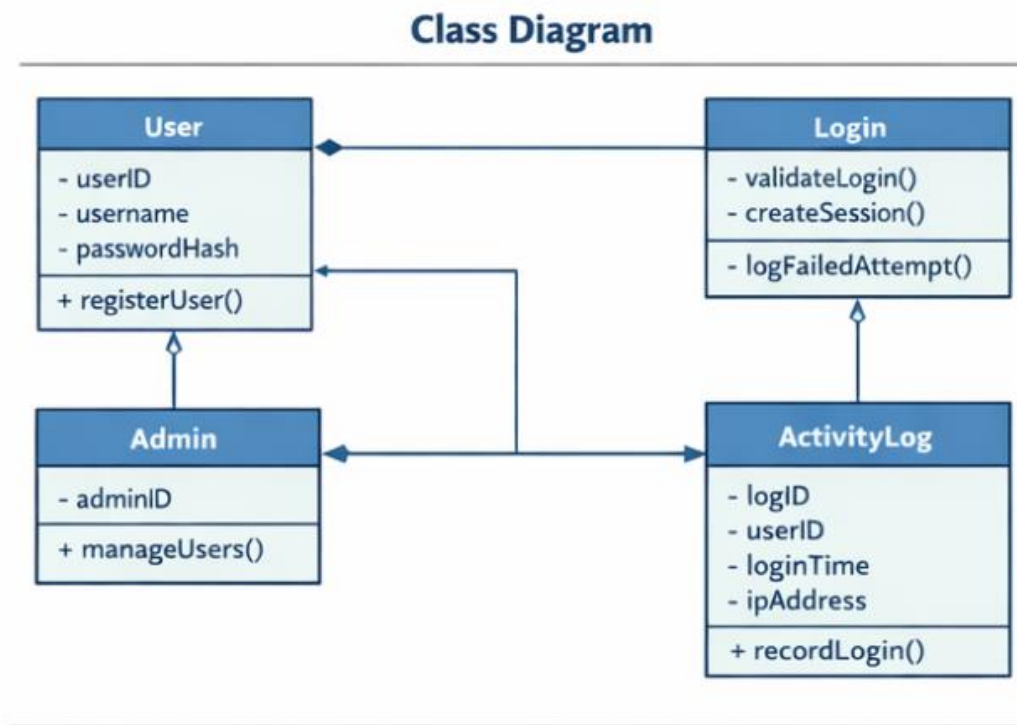


Figure 10.2: Class Diagram

Figure 10.2: Class Diagram

10.3 SEQUENCE DIAGRAM

The **Sequence Diagram** illustrates the **dynamic behavior** of the system by modeling the sequence of operations during a specific scenario, such as a user logging into the system. It shows the interaction between system components over time:

- The **User** enters login credentials on the login page.
- The **Login module** validates the credentials against the database.
- If authentication is successful, the system creates a session and redirects the user to the **User Dashboard**.
- In case of failure, appropriate error messages are displayed, and failed attempts can be logged.

This diagram is crucial for understanding the **step-by-step flow of operations**, ensuring that all processes execute in the correct order and that the system behaves as expected under different scenarios.

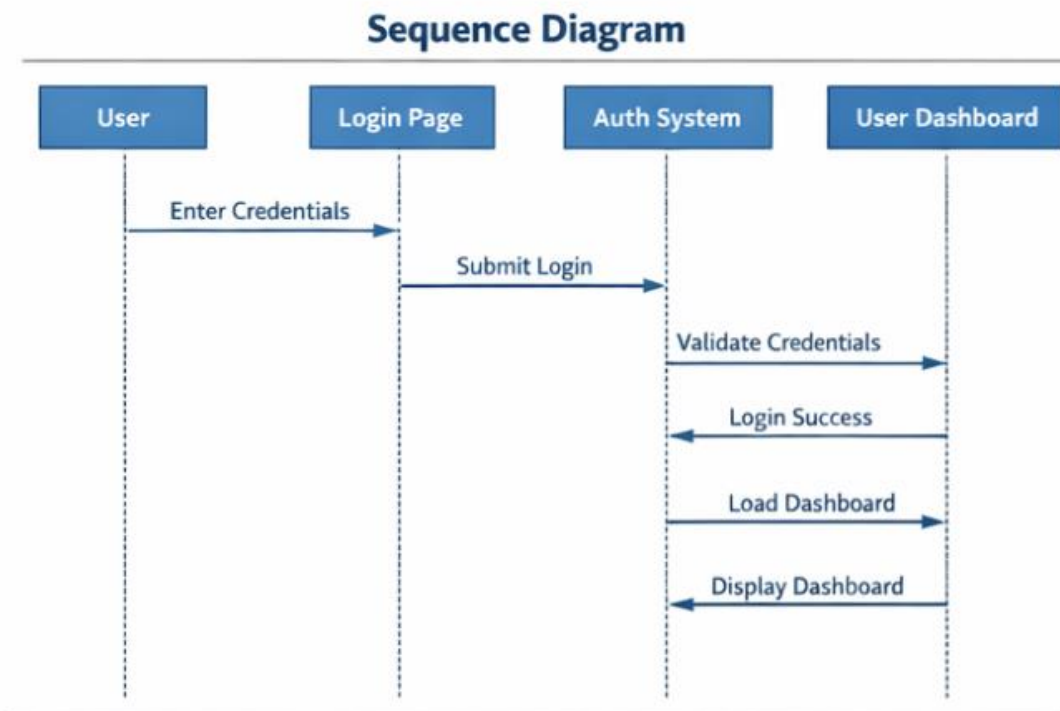


Figure 10.3: Sequence Diagram

Figure 10.3: Sequence Diagram

11.CODING AND IMPLEMENTATION

The **Secure Login System** was developed using **HTML, PHP, and MySQL**. The system leverages HTML forms for user interaction, PHP for server-side processing, and MySQL to store and manage user credentials, activity logs, and system data securely. The implementation emphasizes **security, efficiency, and maintainability**, ensuring that users can interact safely with the system.

11.1 USER REGISTRATION

The **User Registration module** allows new users to create accounts in the system. The workflow includes:

1. **Input Collection:** Users provide details such as username, email, and password via an HTML registration form.
2. **Validation:** The system performs **client-side and server-side validation** to ensure proper input formats and prevent malicious entries.
3. **Password Security:** Before storing in the database, passwords are **hashed using secure algorithms** (e.g., bcrypt) to protect against unauthorized access.
4. **Database Storage:** Validated user details are stored securely in the MySQL database, ensuring data integrity and confidentiality.

This module ensures that only valid and secure user information enters the system.

➤ PHP Code to Handle Registration (register.php):

```
<?php
if($_SERVER['REQUEST_METHOD'] == "POST"){
    $username = $_POST['username'];
    $password = password_hash($_POST['password'], PASSWORD_BCRYPT); // Hash
password

    // Connect to database
    $conn = new mysqli("localhost", "root", "", "secure_login");

    // Insert user into database
    $sql = "INSERT INTO users (username, password) VALUES ('$username', '$password')";
    if($conn->query($sql)){
        echo "Registration successful!";
    } else {
        echo "Error: ".$conn->error;
    }
}
?>
```



Registration Page

Username:

Password:

Role:

11.2 USER LOGIN

The **Login module** authenticates users and manages secure access to the system:

1. **Credential Verification:** Users enter their username and password, which are verified against hashed credentials stored in the database.
2. **Session Management:** On successful authentication, the system creates a **secure session** for the user. This session tracks the user's activity and maintains login state across pages.
3. **Error Handling:** Unauthorized attempts are blocked, and users receive appropriate error messages.
4. **Logging:** All login attempts, successful or failed, are recorded in the **ActivityLog** table to provide accountability and detect suspicious behavior.

This process ensures that **only authorized users** gain access while maintaining detailed audit trails.

➤ PHP Code to Handle Login (login.php):

```
<?php
session_start();
$conn = new mysqli("localhost", "root", "", "secure_login");

if($_SERVER['REQUEST_METHOD'] == "POST"){
    $username = $_POST['username'];
    $password = $_POST['password'];

    $result = $conn->query("SELECT * FROM users WHERE username='$username'");
    if($result->num_rows > 0){
        $user = $result->fetch_assoc();
        if(password_verify($password, $user['password'])){
            $_SESSION['username'] = $username;
            echo "Login successful!";
            header("Location: dashboard.php");
        }
    }
}
```

```
} else {  
    echo "Invalid password!";  
}  
} else {  
    echo "User not found!";  
}  
}  
?>
```



localhost/secure_login/login.php

Login Page

Username:

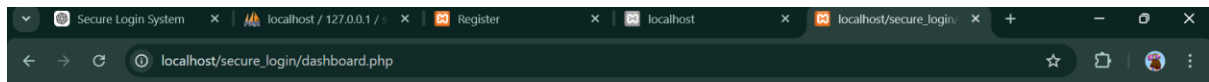
Password:

11.3 DASHBOARD IMPLEMENTATION

The system provides **personalized dashboards** based on user roles:

- **User Dashboard:** Displays user-specific information, such as account details, recent activity, and available actions.
- **Admin Dashboard:** Provides administrators with tools to **manage users, view login activity, and monitor system operations**. It also includes functionality to generate reports and detect unusual activity.

The dashboards are designed with a **user-friendly interface** while ensuring secure access control based on user roles.



User Dashboard

Welcome Pandu

[Logout](#)



Admin Dashboard

Welcome Admin Poojitha

[Logout](#)

11.4 SECURITY FEATURES

The Secure Login System integrates several **essential security mechanisms** to protect against common web attacks:

- **Input Validation:** Prevents **SQL injection**, **cross-site scripting (XSS)**, and other malicious input attacks. Shows the system prevents malicious input.

localhost/secure_login/login.php

☆

Paused

Wrong Password!

Login Page

Username:

Password:

Login

localhost/secure_login/login.php

☆

Paused

User Not Found!

Login Page

Username:

Password:

Login

Secure Login System

localhost / 127.0.0.1 /

Register

localhost

localhost/secure_login/

localhost/secure_login/login.php

☆

Invalid Password

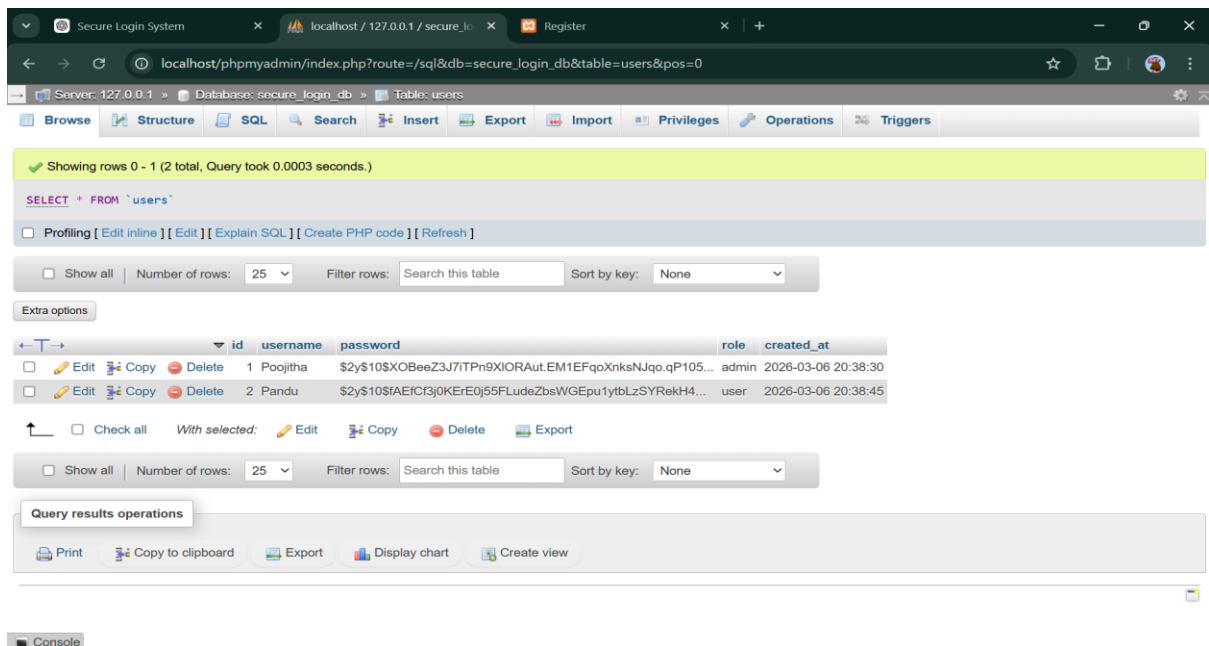
Login

Username:

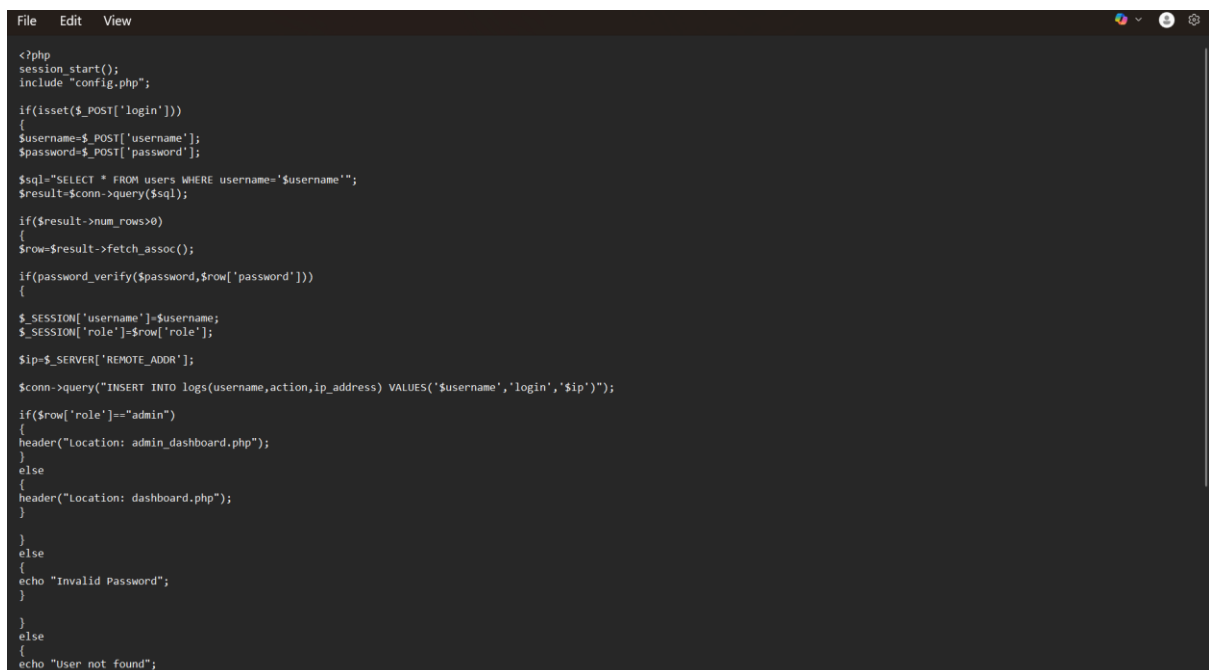
Password:

Login

- **Password Hashing:** Ensures that user passwords are never stored in plaintext. Secure algorithms such as **bcrypt** provide strong encryption. Proves passwords are stored securely.



- **Session Management:** Uses **secure PHP sessions** to prevent session hijacking or unauthorized access. Shows that the system restricts access to logged-in users.



- **Access Control:** Role-based access ensures that users can only access pages and functionality appropriate to their privileges. Shows that only admins can access admin functions.

```

k?php
session_start();
include "config.php";

if(isset($_POST['login']))
{
    $username=$_POST['username'];
    $password=$_POST['password'];

    $sql="SELECT * FROM users WHERE username='$username'";
    $result=$conn->query($sql);

    if($result->num_rows>0)
    {
        $row=$result->fetch_assoc();

        if(password_verify($password,$row['password']))
        {
            $_SESSION['username']=$username;
            $_SESSION['role']=$row['role'];

            $ip=$_SERVER['REMOTE_ADDR'];

            $conn->query("INSERT INTO logs(username,action,ip_address) VALUES('$username','login','$ip')");

            if($row['role']=="admin")
            {
                header("Location: admin_dashboard.php");
            }
            else
            {
                header("Location: dashboard.php");
            }
        }
        else
        {
            echo "Invalid Password";
        }
    }
    else
    {
        echo "User not found";
    }
}

```

- **Activity Logging:** Tracks login attempts and critical actions to support auditing and monitoring. Shows monitoring and accountability of user actions.

Showing rows 0 - 5 (6 total, Query took 0.0006 seconds.)

SELECT * FROM `logs`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	log_id	username	action	login_time	ip_address
☐ Edit ☐ Copy ☐ Delete	1	Poojitha	login	2026-03-06 20:48:34 ::1	
☐ Edit ☐ Copy ☐ Delete	2	Pandu	login	2026-03-06 20:49:36 ::1	
☐ Edit ☐ Copy ☐ Delete	3	Poojitha	login	2026-03-06 20:56:09 ::1	
☐ Edit ☐ Copy ☐ Delete	4	Pandu	login	2026-03-06 20:56:25 ::1	
☐ Edit ☐ Copy ☐ Delete	5	Poojitha	login	2026-03-06 20:56:36 ::1	
☐ Edit ☐ Copy ☐ Delete	6	Pandu	login	2026-03-06 20:56:48 ::1	

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

These features collectively create a robust and secure environment for user authentication and system management.

12. SOFTWARE TESTING

Software testing ensures that the Secure Login System works correctly, provides secure access to authorized users, and protects sensitive data. Testing was done based on **functional and security requirements**, following the principles of **CIA (Confidentiality, Integrity, Availability)** and **AAA (Authentication, Authorization, Accounting)** models.

12.1 TYPES OF TESTING

1. Unit Testing

- Each module of the system was tested individually.
- Examples:
 - User Registration module verifies input and stores hashed passwords correctly.
 - Login module checks credentials and creates sessions.

2. Module Testing

- Ensures that each module (registration, login, dashboards, logout) works as intended.
- Confirms that each module meets both **functional and security requirements**.

3. Integration Testing

- Ensures that all modules work together correctly.
- Example: After registration, a user can log in and access their dashboard successfully.
- Verifies **data flow between modules** and security maintenance.

4. Validation Testing

- Confirms that the system meets project requirements and security objectives.
- Example: Only authorized users can access dashboards; admin privileges are enforced correctly.

5. Output Testing

- Verifies that system outputs are correct and secure.
- Example: Dashboards display the correct user information, and error messages do not reveal sensitive data.

12.2 TEST CASES

1. User Registration

- Enter valid username and password.
- Check that the password is stored in hashed form.

- Attempt registration with an existing username and verify error is shown.

2. User Login

- Enter valid credentials → verify access to user dashboard.
- Enter invalid credentials → verify login is blocked and error message is displayed.

3. Admin Login

- Admin credentials → verify access to admin dashboard.
- Non-admin login → verify admin functions are inaccessible.

4. Session Management

- Access dashboard without logging in → verify redirection to login page.
- Verify session expires on logout or inactivity.

5. Activity Logging

- Check that all login attempts, successful or failed, are recorded in the database.
- Verify that admin can view login history for monitoring purposes.

6. Password Security & Input Validation

- Verify passwords are hashed and not stored in plaintext.
- Test invalid or malicious inputs → system blocks SQL injection and XSS attempts.

13. OUTPUT SCREENS

The Secure Login System provides clear and functional outputs for both **users** and **administrators**, demonstrating the successful implementation of all modules. These outputs highlight the system's compliance with **CIA (Confidentiality, Integrity, Availability)** and **AAA (Authentication, Authorization, Accounting)** models, ensuring secure user authentication and access control.

13.1 USER REGISTRATION MODULE

The User Registration module allows new users to create accounts securely. Users must provide a **username** and **password** to register. The system performs **input validation** to prevent invalid entries and malicious attacks. Passwords are **hashed** before being stored in the database to ensure confidentiality.

Expected Behavior:

- On successful registration, the system displays a “**Registration Successful**” message.
- If a user tries to register with an **already existing username**, an error message is displayed.
- The system prevents SQL injection and invalid inputs.



Figure 13.1: User Registration Confirmation Screen

13.2 USER LOGIN SCREEN

The Login module authenticates users before granting access to the system. Users must enter a valid **username** and **password**.

Expected Behaviour:

- Correct credentials redirect the user to the **User Dashboard**.
- Incorrect credentials display an **error message** without revealing sensitive information.

- All login attempts, successful or failed, are logged in the **Activity Log**.



Figure 13.2: User Login Screen

13.3 USER DASHBOARD SCREEN

After a successful login, users are redirected to the **User Dashboard**, which displays personalized information and available actions.

Expected Behavior:

- Dashboard shows the **logged-in username** and account-related information.
- Provides links for **logout** and other allowed actions.
- Access is restricted to **authenticated users** only.
- Attempts to access the dashboard without logging in redirect to the login page.

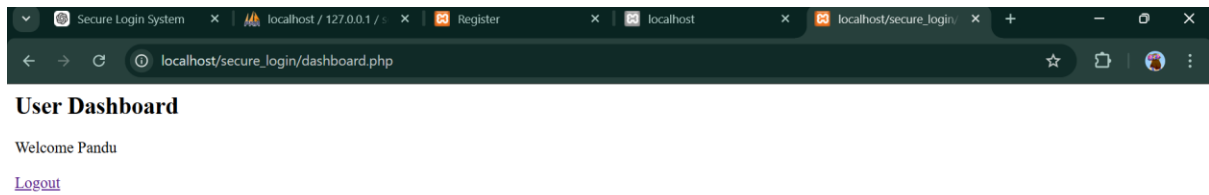


Figure 13.3: User Dashboard

13.4 ADMIN DASHBOARD SCREEN

The Admin Dashboard allows administrators to **monitor user activity** and manage the system. Only users with the **admin role** can access this page.

Expected Behavior:

- Displays **all user login activities**, including successful and failed login attempts.
- Provides tools to **manage users** and monitor system usage.
- Enforces **role-based access control**: normal users cannot access admin features.
- Supports auditing to detect unusual activity.

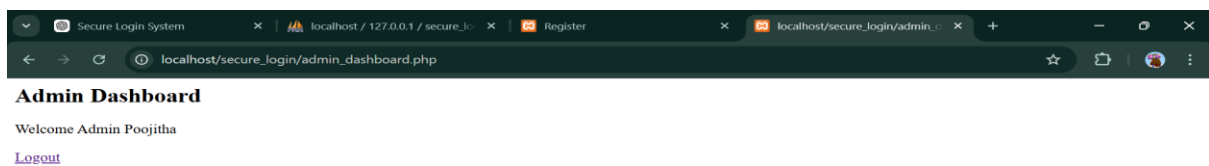


Figure 13.4: Admin Dashboard

13.5 FAILED LOGIN ATTEMPT SCREEN

The system blocks unauthorized access attempts. If a user enters invalid credentials, the system displays a **generic error message**.

Expected Behaviour:

- Prevents login with incorrect username or password.
- Does **not reveal which part is incorrect** (username or password) to protect against attacks.
- Logs all failed login attempts in the Activity Log.

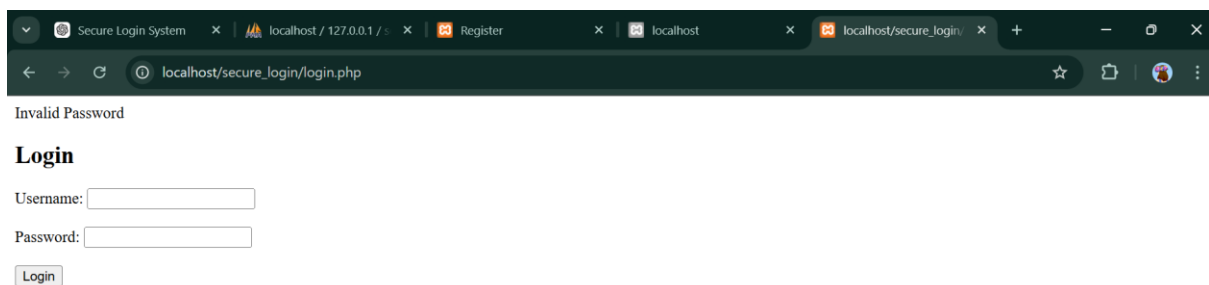


Figure 13.5: Failed Login Attempt Screen

13.6 ACTIVITY LOG SCREEN

The Activity Log records all login attempts and critical user actions. Administrators can view the log to track user activity and detect suspicious behavior.

Expected Behaviour:

- Logs include **username, action performed, and timestamp**.
- Allows admin to audit system access for **accountability and security monitoring**.
- Supports detection of repeated failed login attempts or unusual behaviour.

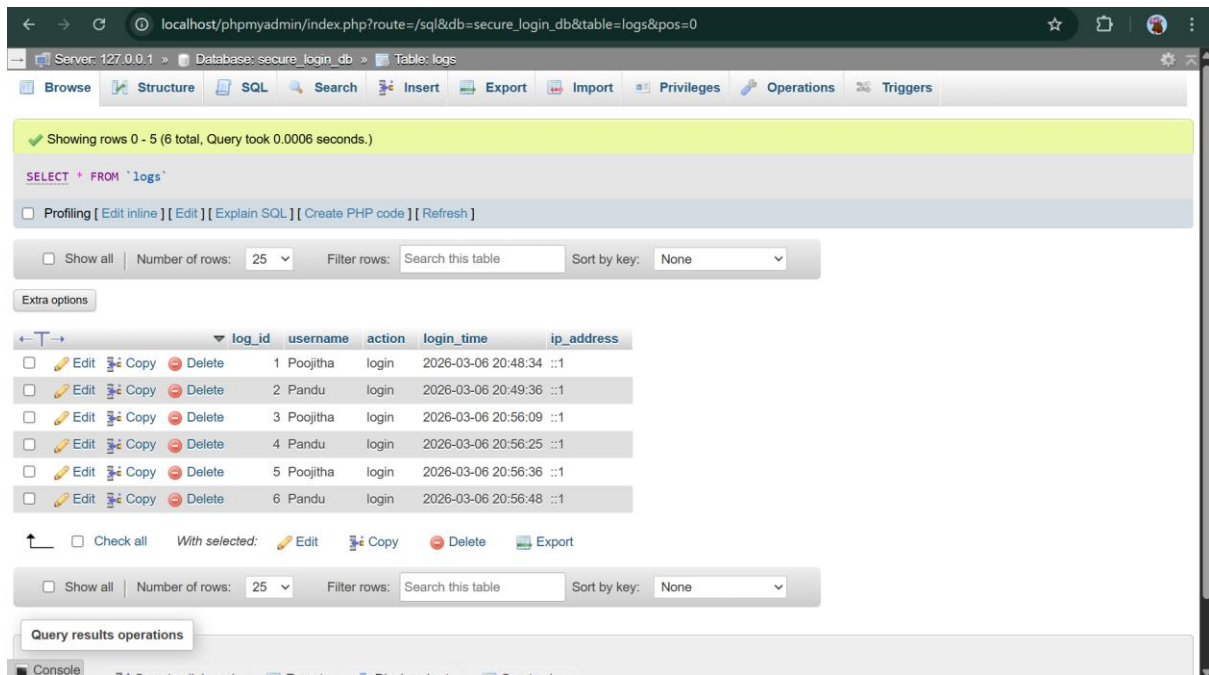


Figure 13.6: Activity Log Screen

13.7 SECURITY VERIFICATION IN SCREEN

All screens also demonstrate **security measures** implemented in the system:

- **Password Hashing:** Ensures that passwords are stored securely.
- **Session Management:** Prevents unauthorized access to dashboards.
- **Input Validation:** Protects against SQL Injection and XSS attacks.
- **Role-Based Access Control:** Ensures only admins can access sensitive screens.

These outputs confirm that the Secure Login System is **fully functional, secure, and reliable**.

14. CONCLUTION

The Secure Login System project successfully demonstrates the development and implementation of a robust authentication system using **HTML, PHP, and MySQL**, integrating key security principles based on the **CIA (Confidentiality, Integrity, Availability) model** and the **AAA (Authentication, Authorization, Accounting) framework**.

Through this project, we achieved the following objectives:

1. Secure User Authentication:

- Only authorized users can access the system.
- Passwords are hashed to prevent unauthorized access.
- Session management ensures user sessions are protected from hijacking.

2. Controlled Access and Authorization:

- Role-based access restricts normal users and admin users to their respective dashboards.
- Sensitive operations and activity logs are accessible only to administrators.

3. Activity Monitoring and Accountability:

- All login attempts, both successful and failed, are recorded in the activity log.
- Administrators can monitor user activity to detect unusual or suspicious behavior.

4. Data Confidentiality and Integrity:

- Input validation prevents SQL injection and XSS attacks, ensuring system reliability.
- Stored data is secure and cannot be modified maliciously by unauthorized users.

5. User-Friendly and Efficient Design:

- The system provides intuitive interfaces for registration, login, dashboards, and administration.
- Screens are designed to be clear and accessible, enhancing the overall user experience.

Overall, this project successfully demonstrates a practical and secure implementation of a login system, balancing both usability and strong security measures. The system can serve as a foundation for larger web applications requiring secure authentication and access control.

15. FUTURE SCOPE

The Secure Login System developed in this project provides a strong foundation for secure user authentication and system monitoring. However, there are several opportunities for enhancement and expansion, which can make the system more advanced, versatile, and suitable for real-world applications:

1. Integration with Multi-Factor Authentication (MFA):

- Implementing MFA (like OTP via email/SMS or authenticator apps) will add an extra layer of security, making it more resistant to account compromise.

2. Enhanced Password Security and Policies:

- Introduce stricter password rules (minimum length, special characters, periodic updates).
- Support password recovery with secure verification methods.

3. Role-Based and Permission-Based Access Control:

- Extend the system to include multiple user roles with granular permissions, allowing more complex organizational structures.

4. Real-Time Monitoring and Alerts:

- Add real-time notifications for suspicious login attempts, unusual activity, or multiple failed logins.
- Alert administrators immediately via email or dashboard notifications.

5. Integration with External Systems:

- The system can be integrated with enterprise applications, e-commerce platforms, or mobile applications to provide a unified authentication service.

6. Logging and Reporting Enhancements:

- Generate detailed reports and analytics on user activity for auditing and compliance.
- Maintain long-term logs with secure storage for regulatory purposes.

7. Security Improvements with Encryption:

- Use HTTPS for secure communication.
- Encrypt sensitive data in the database for additional confidentiality.

8. Scalability and Performance Optimization:

- Optimize the system for handling large numbers of users simultaneously.
- Introduce caching or load-balancing techniques for high-traffic environments.

16.BIBILOGRAFY / PROJECT REFERENCES (LOCAL URLs)

1. **Database Management (phpMyAdmin):** <http://localhost/phpmyadmin>
 - Used to manage the MySQL database for the Secure Login System.
2. **User Registration Page:** http://localhost/secure_login/register.php
 - Allows new users to register securely.
3. **User Login Page:** http://localhost/secure_login/login.php
 - Authenticates users and grants access to the system based on roles.